# Proofline

## Your rules.
## Checked on every change.

A portable test harness: a reusable system for running checks and keeping evidence as your application changes.

# Author once.
# Repeat with confidence.

Turn an important rule into a check the whole team can repeat.

01

## Define the rule

Name the behavior and the result that must stay true.

→ 02

## Write the test

Write and review deterministic tests with clear ownership.

→ 03

## Repeat on changes

Run checks on changes. Retain results and diagnostics.

Continuous integration (CI) reruns these tests automatically. No AI model or agent runs during test execution.

# Small checks. Real journeys.

## 404

### Focused tests · Vitest

Rules, calculations, APIs, permissions, retries and interface behavior.

## 43

### Browser tests · Chromium

Real navigation across screens, plus scenario and containment checks.

Your app → Contracts + runners → Focused + browser checks → Reports + evidence

32 critical rules map to exact tests. Snapshot: 45 unit files across 11 targets (one empty).
The 43 browser tests include 24 scenario tests.

# A deposit must not earn commission

A $25 deposit should earn $0 commission. A controlled source change breaks that rule.
Replay real captured runs from an isolated copy. A developer applies the fix.

## Bug present

$2.50

Wrong commission on the $25 deposit.
Existing test: FAIL · exit 1

```
AssertionError: expected [ { …(8) } ] to deeply equal []
+     "amountCents": 250,
```

## After the developer fix

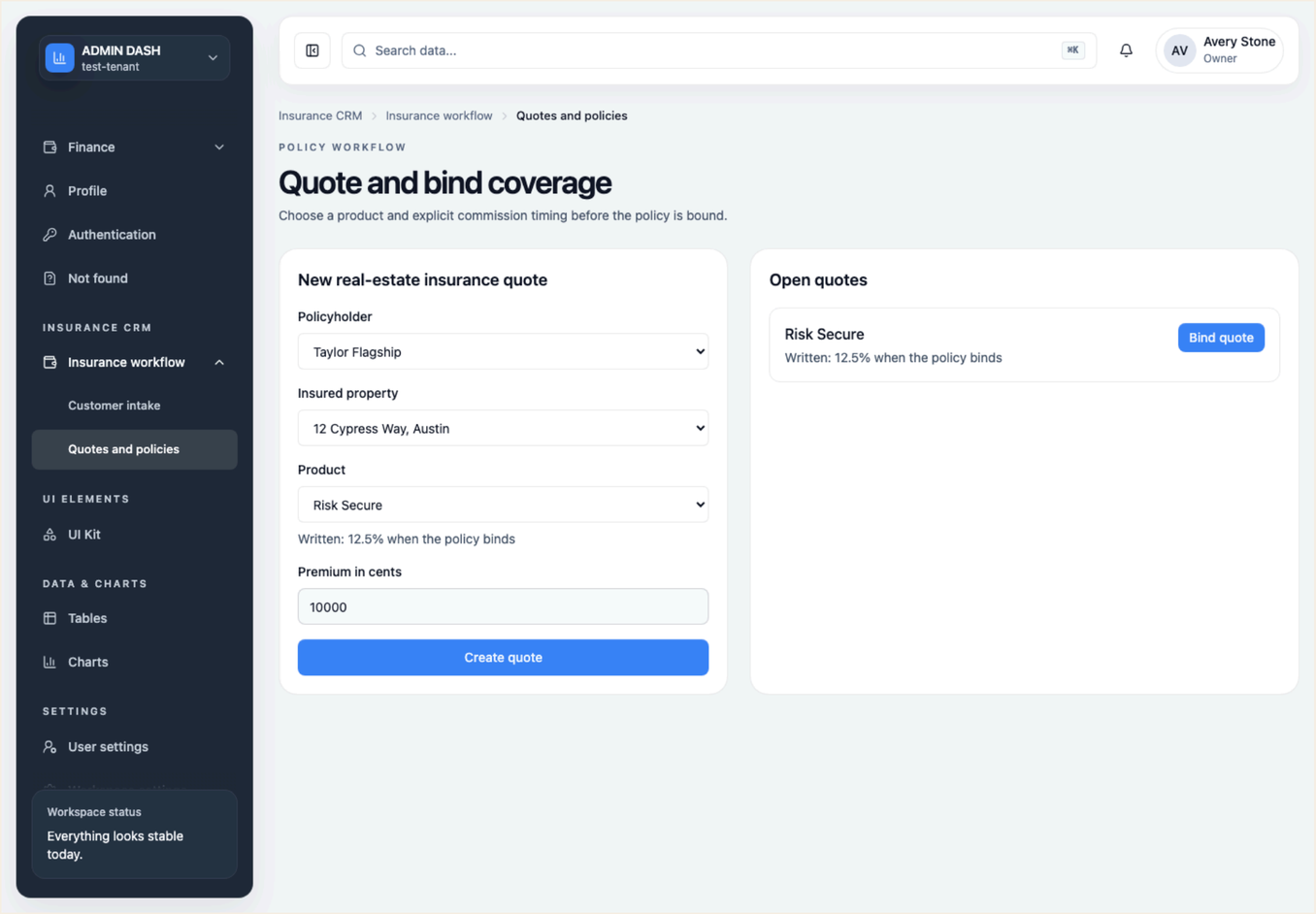
$0.00

Correct commission on the deposit.
Same unchanged test: PASS · exit 0

```
 Test Files  1 passed (1)
      Tests  1 passed | 23 skipped (24)
```

Captured execution, not a live test in this slide. The test was unchanged. The fixed finance suite passed 24 / 24.
Actual run results · Reproduce this controlled regression · Source change

# Follow a real user journey.




01 **Sign in**

A protected link requires login.

02 **Create the customer**

Add their insured property.

03 **Quote and bind**

Use visible controls to create coverage.

04 **Open finance**

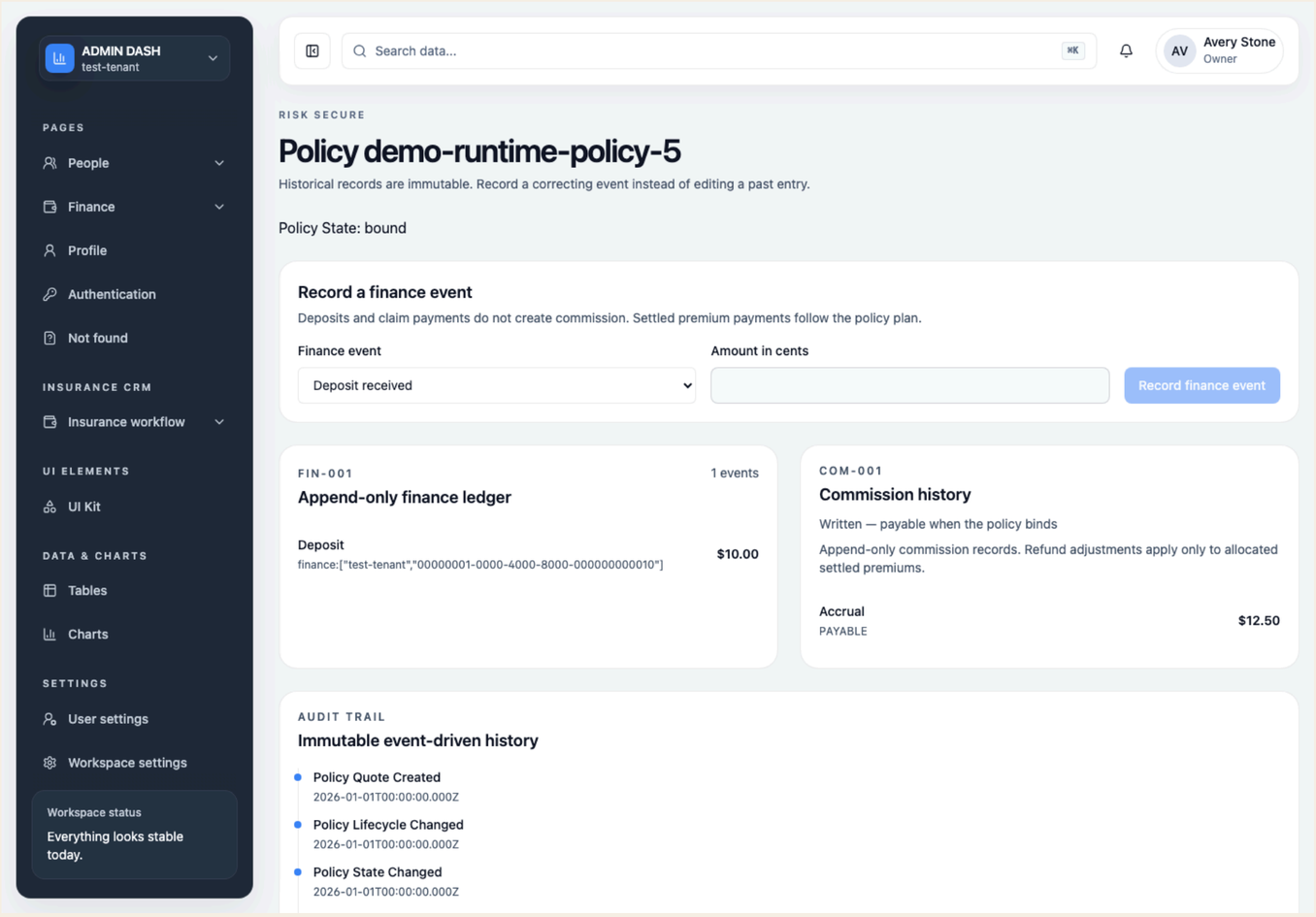
Record a deposit; inspect the ledger.

05 **Log out and revisit**

The protected policy is denied.

Static frame. Local test build of the reference CRM · synthetic data · Play the recording in the browser edition

# The screen is half the proof.




**1** bound policy

**1** finance ledger

**0** pending messages

After the visible actions, the test inspects persisted state.

Logout also revokes the authenticated session.

The recorded run also retained one customer, property, quote, finance entry and commission entry, with zero unexpected requests.

# Trust includes the difficult paths.

01

## Right access

Roles and tenants control which routes and records a user can reach.

02

## Safe recovery

Retrying after a lost response does not duplicate the policy or ledger.

03

## Isolated runs

Fresh state and deterministic fakes. Test adapters stay out of production.

Missing or ambiguous evidence fails closed.

# Coverage you can inspect.

## 8 / 8

### Governed groups meet their gates

Approved minimums can hold or rise.

A passing gate does not mean 100% coverage.

Exercised lines / classified source




12 classified files; covered ÷ total lines. This is a subset, not global coverage. The gate checks each file across four metrics.

Execution safeguards includes runner 0/263 in its parent report; subprocess behavior has separate test evidence.

# A measured local baseline.

Two official passes of the same full Chromium suite.




43 tests · five local workers · reporter durations · builds excluded

Observations from one machine. No speedup or complete-gate timing is claimed.

# Bring the proof to your app.

## Reuse Proofline

Selection · isolated scenarios
Coverage gates · retained evidence



## Connect your application

Routes · business rules · auth
APIs · deterministic provider fakes

Repository-native tooling to adapt for your React/Vite app.
The complete declared suite runs in Chromium. Firefox/WebKit smoke is opt-in.